INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DO CEARÁ - CAMPUS UMIRIM

CURSO DE TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

DISCIPLINA: Programação Web
PROFESSOR: Jardel das Chagas Rodrigues
ALUNOS: Antônio William Brandão Sales & Mikael Moreira Viana

RELATÓRIO TÉCNICO — FITCATALOG

Sistema de Gestão e Catálogo de Exercícios de Musculação

1. Tema Escolhido e Faixas de Validação

O projeto FitCatalog consiste em uma API REST desenvolvida em Node.js com o framework Express para a gestão, consulta e cadastro de exercícios de musculação e suas respetivas cargas recordes/alvo.

Regras de Negócio e Validações Server-Side:

- **Nome do Exercício:** String obrigatória, não vazia, com comprimento mínimo de **3 caracteres**.
- **Grupo Muscular:** String obrigatória (ex: *Peito, Costas, Pernas, Braços*).
- **Faixa de Validação de Carga (Numérica):** A carga deve ser obrigatoriamente um valor numérico compreendido entre **1 kg e 600 kg** (limite baseado nos recordes mundiais absolutos de levantamento de peso). Valores fora desse intervalo ou não numéricos são terminantemente rejeitados com status `400 Bad Request`.

---

2. Instruções de Execução do Projeto

Para executar a aplicação e rodar a suíte de testes localmente, siga os comandos abaixo em ordem no seu terminal:

```
# 1. Clonar o repositório do projeto
git clone https://github.com/mikaelviana-0909/FitCatalog-trabalho-programacao-
web.git

# 2. Entrar no diretório do projeto
cd FitCatalog-trabalho-programacao-web

# 3. Instalar as dependências do Node.js
npm install

# 4. Iniciar o servidor HTTP
node servidor.js

# 5. Em um SEGUNDO terminal, executar a ferramenta de validação
node bancada.js GET /api/exercicios
```

3. Registros das Saídas da Bancada de Testes (bancada.js)

A. Listagem Completa (GET /api/exercicios) — Status 200 OK

```
PS G:\trabalho_pw\musculacao> node bancada.js GET /api/exercicios
REQUISIÇÃO   GET /api/exercicios
```

```
RESPOSTA     200 OK
             connection: keep-alive
             content-length: 247
             content-type: application/json; charset=utf-8
             date: Wed, 28 Oct 2026 13:59:05 GMT
             etag: W/"f7-DdR0wAeq0U7OTYiur0ik+Obdk48"
             keep-alive: timeout=5
             x-powered-by: Express

CORPO
[{"id":1,"nome":"Supino Reto","grupo":"Peito","carga":355},
{"id":2,"nome":"Agachamento Livre","grupo":"Pernas","carga":497},
{"id":3,"nome":"Puxada Frontal","grupo":"Costas","carga":187},
{"id":4,"nome":"Rosca Direta","grupo":"Braços","carga":114}]
```

B. Filtro por Categoria (GET /api/exercicios?grupo=Peito) — Status 200 OK

```
PS G:\trabalho_pw\musculacao> node bancada.js GET /api/exercicios?grupo=Peito
REQUISIÇÃO   GET /api/exercicios?grupo=Peito

RESPOSTA     200 OK
             connection: keep-alive
             content-length: 59
             content-type: application/json; charset=utf-8
             date: Wed, 28 Oct 2026 14:01:17 GMT
             etag: W/"3b-4Li835rWfUtRceOSq2UVU1f66hs"
             keep-alive: timeout=5
             x-powered-by: Express

CORPO
[{"id":1,"nome":"Supino Reto","grupo":"Peito","carga":355}]
```

C. ID Inexistente (GET /api/exercicios/99) — Status 404 Not Found

```
PS G:\trabalho_pw\musculacao> node bancada.js GET /api/exercicios/99
REQUISIÇÃO   GET /api/exercicios/99

RESPOSTA     404 Not Found
             connection: keep-alive
             content-length: 47
             content-type: application/json; charset=utf-8
             date: Wed, 28 Oct 2026 14:02:03 GMT
             etag: W/"2f-4AB/wJK5UiRt4xTg7jBoeuavswY"
             keep-alive: timeout=5
             x-powered-by: Express
```

```
CORPO
{"erro":"Exercicio nao encontrado no sistema."}
```

D. Cadastro Válido (POST /api/exercicios) — Status 201 Created

```
PS G:\trabalho_pw\musculacao> node bancada.js POST /api/exercicios
nome="Levantamento Terra" grupo="Costas" carga=501

REQUISIÇÃO   POST /api/exercicios
             corpo: {"nome":"Levantamento Terra","grupo":"Costas","carga":501}

RESPOSTA     201 Created
             connection: keep-alive
             content-length: 65
             content-type: application/json; charset=utf-8
             date: Wed, 28 Oct 2026 14:02:40 GMT
             etag: W/"41-sJkFt4nBF8s+lIKnWvXixWQsFFE"
             keep-alive: timeout=5
             x-powered-by: Express

CORPO
{"id":5,"nome":"Levantamento Terra","grupo":"Costas","carga":501}
```

E. Cadastro Inválido por Carga Fora da Faixa (POST /api/exercicios) — Status 400 Bad Request

```
PS G:\trabalho_pw\musculacao> node bancada.js POST /api/exercicios nome="Supino
Mutante" grupo="Peito" carga=750
REQUISIÇÃO   POST /api/exercicios
             corpo: {"nome":"Supino Mutante","grupo":"Peito","carga":750}

RESPOSTA     400 Bad Request
             connection: keep-alive
             content-length: 63
             content-type: application/json; charset=utf-8
             date: Wed, 28 Oct 2026 14:03:21 GMT
             etag: W/"3f-go55Yv4aLhzNY/EjQfjNCpN7aN8"
             keep-alive: timeout=5
             x-powered-by: Express

CORPO
{"erro":"A carga deve ser um numero valido entre 1kg e 600kg."}
```

4. Análise Crítica e Resposta Teórica (Questão de Fixação)

> **"Não preciso de back-end, faço tudo em JavaScript no navegador, inclusive acessar o banco de dados."**

A afirmação do colega é conceitualmente incorreta e representa um risco gravíssimo para qualquer aplicação web real. Acessar o banco de dados diretamente pelo navegador ignora a arquitetura cliente-servidor e gera pelo menos dois problemas críticos:

1. **Exposição Inevitável de Credenciais e Quebra de Segurança:** Para que o navegador se conecte diretamente a um banco de dados, as credenciais de acesso (usuário, senha, chave de API e string de conexão) teriam que ficar salvas no arquivo JavaScript enviado ao cliente. Como o código do front-end é público, qualquer pessoa abrindo o *DevTools* (F12) do navegador poderia extrair essas chaves e manipular, roubar ou apagar todas as tabelas do banco sem qualquer barreira de proteção.

2. **Incapacidade de Impor Regras de Negócio e Integridade dos Dados:** Validações feitas no navegador podem ser facilmente burladas, seja desativando o JavaScript, alterando o código na aba *Elements* ou enviando requisições diretas por ferramentas de teste. **Evidenciamos isso concretamente na atividade no uso da `bancada.js`:** quando enviamos uma requisição `POST` simulando um cadastro com carga de 750 kg (que violava a regra da aplicação), o formulário web foi totalmente passado. Se não houvesse o back-end para interceptar o pedido e responder com `HTTP 400 Bad Request`, o banco de dados teria aceito um dado corrompido/inválido.

Portanto, o back-end é insubstituível para atuar como uma **camada de segurança server-side**, protegendo as credenciais da aplicação e garantindo que apenas dados sanitizados e autorizados cheguem ao banco de dados.